

Feature relevancy, necessity and usefulness: complexity and algorithms

Tomás Capdevielle
Santiago Cifuentes⁰

ICC Conicet, Universidad de Buenos Aires

TOMAS.CAPDEVIELLE@GMAIL.COM
CIFUENTESSANTIAGO76@GMAIL.COM

Abstract

Given a classification model and a prediction for some input, there are heuristic strategies for ranking features according to their importance on regards to the prediction. One common approach to this task is rooted in propositional logic and the notion of *sufficient reason*. Through this concept, the categories of relevant and necessary features were proposed in order to identify the crucial aspects of the input. This paper improves the techniques and algorithms known for deciding which are the relevant and/or necessary features, showing in particular that necessity can be detected efficiently in complex models such as neural networks. We also generalize the notion of relevancy and study associated problems. Moreover, we present a new global notion (i.e. that intends to explain whether a feature is important for the behavior of the model in general) of *usefulness* and prove it to be related with relevancy and necessity. Furthermore, we develop efficient algorithms for detecting it in decision trees and other more complex models.

1. Introduction

Artificial Intelligence (AI) has rapidly become an integral part of our daily lives, with applications of AI and machine learning models becoming increasingly popular (Brynjolfsson & McAfee, 2014). Thanks to the greater affordability of computing resources (McCallum, 2023) and the availability of vast public data repositories (Taylor, 2024), these models can now be trained more extensively, achieving levels of performance that were once unimaginable (Brynjolfsson & Mitchell, 2017; Lu, 2019; Ng, 2016; Shao et al., 2022).

On the one hand, these advancements in model performance have led to impressive developments in generative AI and more precise results in classification models. On the other hand, as both the power and popularity of AI models have grown, several concerns regarding their applications have also arisen due to multiple reasons including machine bias, catastrophic failures involving AI models, and ethics (Angwin et al., 2022; Gunning & Aha, 2019; Ribeiro et al., 2016; Tjoa & Guan, 2020).

As the complexity and depth of modern AI systems increases, the ability of humans to *understand* and *interpret* their behaviour diminishes (Narodytska et al., 2018; Ruan et al., 2018). Thus, the emergent field of XAI (eXplainable AI) aims to develop techniques and heuristics to evaluate and explain the decisions and outputs of the models. Such tools are crucial in applications of a sensitive nature, such as AI-assisted medical diagnosis and credit scoring. Explaining AI-driven decisions is a key component in advancing toward the broad goal of an ethical use of AI, according to recent initiatives such as in European Commission, 2018; Goodman and Flaxman, 2017; Madiega, 2019. Moreover, the use of XAI tools extends

⁰. Corresponding author

beyond enhancing the security of AI application: by providing explanations for the results obtained by these models, XAI enables a deeper understanding of their behavior, which can aid in detecting problems and identifying potential areas for improvement.

One way of providing insight into a prediction of an input consists in ranking features according to their importance. Most common *feature ranking* methods are grounded in game theory, such as the Shapley values (Shapley et al., 1953), and therefore possess theoretical properties desirable for the explainability task. However, computing the exact Shapley values is usually computationally expensive (Arenas et al., 2023; Marzouk et al., 2025; Van den Broeck et al., 2022), which is why approximations, such as the SHAP score (Lundberg & Lee, 2017) are often employed in practice. These approximations might be practical, but they are less accurate (Fryer et al., 2021) and do not maintain all the theoretical properties.

Both the Shapley values and its approximations have been pointed out for assigning non-zero score to features that are irrelevant from a logical perspective, and similarly assign zero score to relevant ones (Huang & Marques-Silva, 2023). *Relevancy*, in the logical context, is defined upon the notion of sufficient reason or abductive explanation (Marques-Silva, 2023). Intuitively, a set of features S is a sufficient reason for some prediction if they determine the prediction by themselves: that is, changing the other features' values would not change the model result, as long as the values from the features from S remain unchanged. Then, relevant features are defined as those features present in some sufficient reason, while necessary ones are defined as those belonging to all of them (Audemard et al., 2021).

Detecting relevancy is hard in general, and thus the problem is usually considered in the context of simple models such as decision trees or restricted binary decision diagrams (Audemard et al., 2020; Huang et al., 2023). While one might argue that these families of classifiers are too simplistic to be useful in practical environments, we observe that: first, they provide a valuable starting point for studying the feasibility of logic-based explainability in a world increasingly dominated by black-box systems; and second, it is possible to *compile* complex systems into these simpler models in an *off-line* manner to then perform fast *online* explainability queries (Bertossi & León, 2023; Darwiche et al., 2004; De Colnet & Mengel, 2020; Huang, Izza, Ignatiev, Cooper, et al., 2021; Marquis, 2015),

Our contributions In this paper we extend previous results on the complexity of detecting feature relevancy and necessity, and moreover provide efficient algorithms to solve these problems for families of models where the complexity was unknown. In particular, we show that relevancy can be detected in decision trees with numerical features, extending the result for general trees with categorical features (Huang, Izza, Ignatiev, Cooper, et al., 2021). For the case of necessity, we extend the tractability frontier showing that it can be detected in any binary decision model, and even provide linear time algorithms to compute all necessary features for decision trees and FBDDs. We also extend the notion relevancy in two ways, show that they both are NP-COMplete to compute for decision trees, and provide efficient algorithms for restricted cases.

Finally, we propose a global notion to decide which features are important for the model in general, which we call *usefulness*. We show that it is related to the notions of relevancy and necessity, and that the complexity of detecting it is related to the problem of deciding whether two models are equivalent. We also define a scoring notion based on it, and provide a general algorithm for computing it. In particular, the proposed procedure

runs in quadratic time for decision trees. We compute this score for three different datasets and show that the ranking it induces is consistent with other scoring schemes.

Related work The problem of computing relevant and necessary features was introduced in (Audemard et al., 2021) and solved for the case of binary decision trees with boolean features (i.e. decision trees classifying each input into one of two classes, where each feature of the input can be either 0 or 1). In Huang, Izza, Ignatiev, and Marques-Silva, 2021 these results were extended for decision trees with generic categorical features and an arbitrary number of classes. More recently, in Huang et al., 2023 hardness for detecting relevancy was shown for FBDDs, and polynomial time algorithms were developed to compute the set of necessary features for families as expressive as D-DNNFs. In Darwiche and Ji, 2022 a decision tree model admitting both categorical and numerical features similar to ours was considered, but they do not study the problem of relevancy and necessity.

Other problems related to computing sufficient reasons were considered in the literature. In particular, Audemard et al., 2023 studied these problems for Boosted Trees, and Carbonnel et al., 2023 for multivariate decision trees. In Marques-Silva, 2023 the complexity of enumerating prime implicants of D-DNNFs was studied. Izza et al., 2020 made a more experimental study on computing these type of explanations for decision trees, comparing it with the “direct reason” given by the classification path. Both Audemard et al., 2023 and Huang, Izza, Ignatiev, and Marques-Silva, 2021 consider the problem of computing contrastive explanations.

We observe that Bertossi et al., 2020 proposed a scoring schema based on the notion of *counterfactual cause* (Halpern & Pearl, 2005) which is somewhat related to our notion of usefulness. As we will see, the usefulness of a feature x is related to the number of entities that have a counterfactual cause on x .

Organization In Section 2 we present all definitions that we will consider in the rest of the work, and in particular we present our decision tree model admitting both categorical and numerical features. In Section 3 we present some auxiliary results related to hitting sets that we need for Section 4 where we show our main results and algorithms. In Section 5 we experiment with three datasets to analyze the utility of our scoring proposal. Finally, in Section 6 we state some conclusions and propose future lines of work.

2. Definitions

When considering decision trees, we will work with categorical and numerical features, in a similar manner to Darwiche and Ji, 2022, but treating numerical features as first-class citizens: some of our results can be seen as generalizations of the well-known hitting set dualization results (Liffiton & Sakallah, 2008; Reiter, 1987) already considered in Darwiche and Ji, 2022; Huang, Izza, Ignatiev, and Marques-Silva, 2021 to handle general categorical and numerical features. In addition, all the models that we consider will classify each entity into one of finite categories.

Let X be some finite set of elements, that we will consider as a set of features, and that we can partition as $X = X_C \cup X_N$, where X_C denotes the set of categorical features and X_N the set of numerical ones. For each categorical feature $x \in X_C$ we consider given some domain $\mathcal{D}_x$ such that $|\mathcal{D}_x| < \infty$. We say that a categorical feature is binary if $|\mathcal{D}_x| = 2$. Meanwhile, for each numerical feature $x \in X_N$ we consider given some domain

$\mathcal{D}_x = [m_x, m_x + 1, \dots, M_x]$, where $m_x \in \mathbb{R} \cup \{-\infty\}$ denotes the smallest value that the feature can take, while $M_x \in \mathbb{R} \cup \{\infty\}$ denotes the biggest one. We define the set of entities over X as the set $\mathbf{ent}(X) = \{f : X \rightarrow \bigcup_{x \in X} \mathcal{D}_x : f(x) \in \mathcal{D}_x \forall x \in X\}$. Given an entity $e \in \mathbf{ent}(X)$ and some feature $x \in X$ the expression $e(x)$ indicates the value that feature x has for entity e . We say that a model is binary if it only uses binary categorical features.

A k -class model M over X is a mapping from $\mathbf{ent}(X)$ to $\{0, 1, \dots, k-1\}$ ¹. Given an entity $e \in \mathbf{ent}(X)$ the value $M(e)$ indicates the class to which e belongs according to M . Given two models M_1 and M_2 we use $M_1 \equiv M_2$ to indicate that $M_1(e) = M_2(e)$ for all entities $e \in \mathbf{ent}(X)$. We refer to 2-class models as *Boolean* classifiers, and we say that a boolean model M *accepts* an entity e if $M(e) = 1$ and *rejects* it otherwise.

Given some subset $S \subseteq X$ of features and some entity $e \in \mathbf{ent}(X)$ we define the set of features consistent with e on S as $\mathbf{cw}(e, S) = \{e' \in \mathbf{ent}(X) : e'(x) = e(x) \forall x \in S\}$. Given some entity e we denote as $e_{x=b}$ with $x \in X$ and $b \in \mathcal{D}_x$ the unique entity satisfying $e(y) = e_{x=b}(y)$ for all $y \in X \setminus \{x\}$, and $e_{x=b}(x) = b$. Given a model M over X , a feature $x \in X$ and $b \in \mathcal{D}_x$ we define $M_{x=b}$ as the model satisfying $M_{x=b}(e) = M(e_{x=b})$.

A literal l over features X is an expression of one of the following forms:

1. If $x \in X_C$, then $l = x \text{ op } D$ where $\text{op} \in \{\in, \notin\}$ and $D \subseteq \mathcal{D}_x$.
2. If $x \in X_N$, then $l = x \text{ op } b$ where $\text{op} \in \{\leq, >\}$ and $b \in \mathcal{D}_x$.

We denote the negation of a literal l as $\neg l$ and define it in the usual manner by flipping the operator involved in the literal (i.e. $\in$ changes to $\notin$, $\leq$ to $>$, etc).

A *term* is a conjunction of literals, and a *clause* is a disjunction of literals. The size of a term or a clause is defined as its number of literals. A DNF formula is a disjunction of terms, and a CNF formula is a conjunction of clauses. The size of a DNF or CNF formula is defined as the sum of the sizes of its terms or clauses, respectively. Any term $l = x \text{ op } v$ can be understood as a boolean classifier: given any entity $e \in \mathbf{ent}(X)$ we consider that $l(e) = 1 \iff e(x) \text{ op } v$. This can be naturally extended to terms and clauses, and furthermore to DNF and CNF formulas. Whenever the feature x is binary, we will write x to denote the literal $x \in \{1\}$ and $\bar{x}$ to refer to $x \in \{0\}$.

Example 1. *The formula*

$$\varphi(x_1, x_2, x_3, x_4, x_5) = (x_1 \vee \bar{x}_2 \vee x_5) \wedge (x_2 \vee x_3 \vee x_4) \wedge (\bar{x}_2 \vee x_4 \vee \bar{x}_5) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee x_5)$$

is a CNF formula, which can be understood as a boolean model with binary features. Given the entity $e = \{x_1 : 0, x_2 : 0, x_3 : 1, x_4 : 1, x_5 : 0\}$, it holds that $\varphi(e) = 1$.

We say that a boolean class of models $\mathcal{M}$ is closed under...

- *Conditioning* if whenever $M \in \mathcal{M}$, it is the case that $M_{x=b} \in \mathcal{M}$ for every $x \in X$ and $b \in \mathcal{D}_x$, and $M_{x=b}$ can be computed from M , x and b in polynomial time.

1. Whenever the context makes it clear, we will omit specifying the feature set over which the model or the entities are defined

- *Disjoint disjunction* if whenever $M_1, M_2 \in \mathcal{M}$, it is the case that $M = (M_1 \wedge x) \vee (M_2 \wedge \bar{x}) \in \mathcal{M}$, where x is a fresh binary categorical feature not used by neither M_1 or M_2 , and M can be computed efficiently given M_1 and M_2 .
- *Negation* if whenever $M \in \mathcal{M}$ it holds that $\neg M \equiv 1 - M \in \mathcal{M}$ and $\neg M$ can be computed efficiently given M .
- *Conjunction* if whenever $M_1, M_2 \in \mathcal{M}$ it holds that $M = M_1 \wedge M_2 \in \mathcal{M}$ and M can be computed efficiently given M_1 and M_2 .

In this work, we will be interested in *sufficient reasons*:

Definition 1 (Sufficient Reason Abductive Explanation (Marques-Silva, 2023)). *Given a model M and an entity e we define the set of reasons for the prediction $M(e)$ as*

$$SR(M, e) = \{S \subseteq X : M(e') = M(e) \forall e' \in cw(e, S)\}$$

A sufficient reason is a reason that is minimal with respect to the property of being a reason.

Sufficient reasons are minimal subsets of features such that, if their values are preserved as stated by e , then the behavior of the model M does not change, independently of any modification done to the other features.

Example 2. *Consider the model φ and the entity e from Example 1. Then, it holds that $SR(\varphi, e) = \{\{x_2, x_3\}, \{x_2, x_4\}\}$.*

A model may have many sufficient reasons (potentially an exponential number relative to its size as shown in Audemard et al., 2021), and thus any individual reason might not be useful for the explanatory task. Nevertheless, we can distinguish important features by considering those that belong to some or all sufficient reasons. These correspond to the notions of relevant and necessary features, respectively.

Definition 2 (Relevant features). *Given a model M , a feature $x \in X$ and an entity $e \in \mathbf{ent}(X)$ over M we say that x is relevant for the prediction $M(e)$ if there is a sufficient reason $S \in SR(M, e)$ such that $x \in S$.*

Definition 3 (Necessary features). *Given a model M , a feature $x \in X$ and an entity $e \in \mathbf{ent}(X)$ over M we say that x is necessary for the prediction $M(e)$ if for all sufficient reasons $S \in SR(M, e)$ it is the case that $x \in S$.*

The two previous notions are local, i.e. relevant and necessary features can be used to understand the behavior of model M for a particular prediction of some entity e . We introduce a more global notion intended to capture those features that the model uses to distinguish at least two entities.

Definition 4 (Useful features). *Given a model M and a feature $x \in X$ we say that x is useful if there is some $e \in \mathbf{ent}(X)$ and $b \in \mathcal{D}_x$ such that $M(e) \neq M(e_{x=b})$.*

Intuitively, a feature that is not useful is not required by the model because there is not a single entity whose classification depends on the value of that feature. This intuition is formally correct, since if x is not useful for model M then $M \equiv M_{x=b}$ for any $b \in \mathcal{D}_x$. We will show that the notion of usefulness is related to relevancy and necessity (see Proposition 22). We propose the idea of counting the number of entities for which x is useful to assign a value to the feature x in order to rank feature importance.

Example 3. Consider again the model φ and the entity e from Example 1. Observing the sufficient reasons already computed in Example 2, it holds that x_2 is a necessary feature for $\varphi(e)$ while x_3 and x_4 are relevant.

Furthermore, x_1 is not useful for φ (note that the first clause can be simplified with the last one), while the other features are all useful.

In the following we present all families of models that we will use in our results, which have been used as compilation languages for more complex models (Darwiche & Marquis, 2002) and are common in the literature of XAI (Arenas et al., 2023; Barceló et al., 2020; Huang et al., 2022).

The simplest family is the one corresponding to decision trees. Our definition is somewhat non-standard because we allow for numerical and categorical features together.

Definition 5 (Decision tree). A k -class decision tree over features X is a binary tree T (i.e. each node except for the leaves has two children), each of whose internal nodes is labeled with an element from $\{(x, D) : x \in X_C, D \subseteq \mathcal{D}_x\}$ (in which case we call it a categorical node) or rather from $\{(x, b) : x \in X_N, b \in \mathcal{D}_x\}$ (in which case it is a numerical node); and leaves are labeled with a class in the range $\{0, \dots, k-1\}$. We assume the set of children of each internal node v is represented by an ordered list $\{w_1, w_2\}$, and therefore we can refer to w_1 as the “left” child and to w_2 as the “right” child. The size of a decision tree T is defined as its number of nodes, and noted as $|T|$.

Given any categorical node v we denote as $\mathbf{feat}(v) \in X_C$ and $\mathbf{values}(v) \subseteq \mathcal{D}_{\mathbf{feat}(v)}$ the feature and set of values of its label, respectively. We associate a literal $l(v)$ to each categorical node v defined as $l(v) \equiv \mathbf{feat}(v) \in \mathbf{values}(v)$. Given any numerical node v in T , we denote as $\mathbf{feat}(v) \in X_N$ and $\mathbf{value}(v) \in \mathcal{D}_{\mathbf{feat}(v)}$ the feature and value of its label, respectively. We associate a literal $l(v)$ to each numerical node v defined as $l(v) = \mathbf{feat}(v) \leq \mathbf{value}(v)$.

Given $e \in \mathbf{ent}(X)$ we denote the prediction of T for e as $T(e)$, and define it as the label of the leaf reachable from the root using the following strategy: when on a internal node v move to the left child if $l(v)(e) = 1$, and move to the right child otherwise.

See Figure 1 for an example of a decision tree. Without loss of generality, we assume that there are no categorical nodes with labels of the form $(x, \emptyset)$ or $(x, \mathcal{D}_x)$, and similarly that there are no numerical nodes with labels of the form (x, M_x) . If v is a node from T , we refer by T_v to the decision (sub)-tree rooted at v . We denote the class of decision trees as DT, while we denote the class of boolean binary decision trees as DT_B.

One of our algorithms will involve FBDDs, and thus we lay out some definitions.

Definition 6 (FBDD (Free Binary Decision Diagram)). A decision diagram is a boolean binary model given by a DAG (directed acyclic graph) D with a unique node with an in-degree

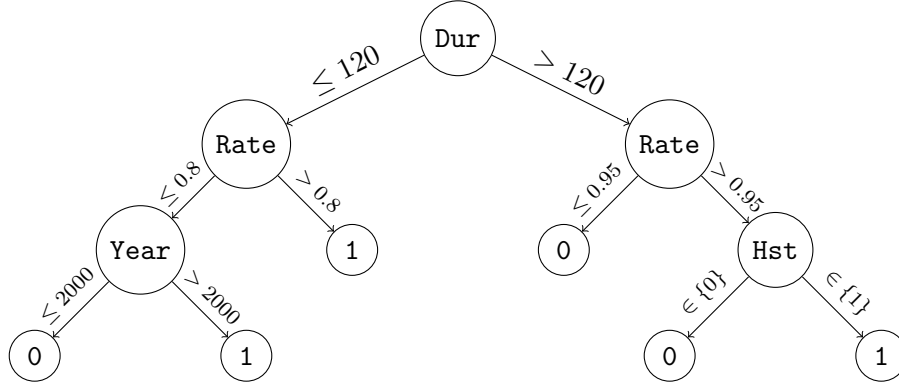


Figure 1: Example of a decision tree representing a recommendation system for a film database. The set of features is **Dur** (duration), **Rate**, **Year** and **Hst** (whether the film is of the historical genre). The domains are $\mathcal{D}_{\text{Dur}} = [0, \infty]$, $\mathcal{D}_{\text{Rate}} = [0, 1]$, $\mathcal{D}_{\text{Year}} = [1888, \infty]$ and $\mathcal{D}_{\text{Hst}} = \{0, 1\}$. Note that the first three features are numerical, while the last one is categorical. The tree classifies entity $e = \{\text{Dur} : 90, \text{Rate} : 0.85, \text{Year} : 2005, \text{Hst} : 0\}$ to 1. Moreover, the sufficient reasons for the result are $\{\text{Dur}, \text{Rate}\}$ and $\{\text{Dur}, \text{Year}\}$, and therefore **Dur** is a necessary feature while **Rate** and **Year** are both relevant.

of 0 identified as the root, and such that each node has an out-degree equal to either 2 (the internal nodes) or 0 (the leaves). Each internal node has its set of reachable nodes ordered as $N(v) = \{w_1, w_2\}$ and thus we can refer to the left and right child of v as $w_1 = \text{left}(v)$ and $w_2 = \text{right}(v)$. Each internal node v has an associated feature $\text{feat}(v)$, and each leaf has an associated label 0 or 1.

Given an entity $e \in \text{ent}(X)$ (where all features are binary) we denote the prediction of D for e as $D(e)$, and define it as the label of the leaf reachable from the root using the following strategy: when on an internal node v move to $\text{left}(v)$ if $e(\text{feat}(v)) = 0$, and move to $\text{right}(v)$ otherwise.

A decision diagram D is an FBDD if every directed path $P = v_0, \dots, v_k$ in D satisfies that $\text{feat}(v_i) \neq \text{feat}(v_j)$ for $0 \leq i < j \leq k$ (i.e. features are not repeated in any path). This is usually referred to as the read-once property.

See Figure 2 for an example of an FBDD. If D is an FBDD and v is a node from D , we use D_v to denote the FBDD obtained by taking v as the root of D .

The other models we mention are standard and their particular details are not needed for the development of this work. These encompass OBDDs (Ordered Binary Decision Diagrams), D-DNNFs and general DNFs (Darwiche & Marquis, 2002). All of them are understood as particular cases of boolean binary models. Moreover, all classes mentioned are closed under conditioning and disjoint disjunction, all binary diagrams are closed under negation, and trees are closed under conjunction.

Any decision tree can be represented as a DNF formula of small size.

Definition 7 (CNF of a boolean decision tree). *Let T be a boolean decision tree. We define its set of paths as $P_T = \{v_0, v_1, \dots, v_k : v_0 \text{ is the root of } T, v_k \text{ is a leaf and } v_{i+1} \text{ is a child of } v_i \text{ for } 0 \leq i < k\}$. Given a path $v_0, \dots, v_k = p \in P_T$ we associate a term F_p to p as*

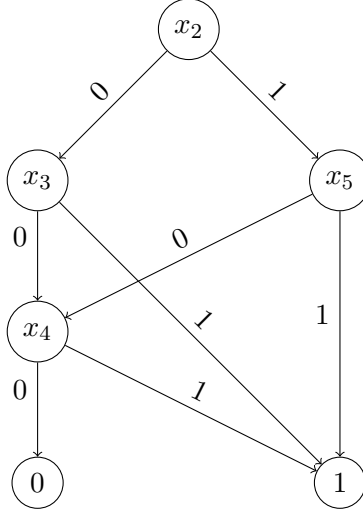


Figure 2: An FBDD representing the CNF formula from Example 1. Note that each directed path does not contain two nodes with the same feature.

$$F_p = \bigwedge_{\substack{i=0 \\ v_{i+1} \text{ is left child of } v_i}}^k l(v_i) \wedge \bigwedge_{\substack{i=0 \\ v_{i+1} \text{ is right child of } v_i}}^k \neg l(v_i)$$

and likewise a clause $C_p = \neg F_p$ (where negation propagates following the De Morgan laws).

Let $P_T^0 \subset P_T$ be the subset of paths of T that end at a node with label 0. Then, it holds that $T \equiv \neg \bigvee_{p \in P_T^0} F_p = \bigwedge_{p \in P_T^0} C_p$. We denote the boolean formula $\bigwedge_{p \in P_T^0} C_p$ as $\mathbf{cnf}(T)$. It holds that $|\mathbf{cnf}(T)| = O(|X||T|)$.

Even though any decision tree is represented by a succinct (i.e. with at most a polynomial overhead in size) CNF formula, it is not the case that any CNF formula can be represented succinctly by a decision tree. Nevertheless, it is a well known fact that any model with a small set of positive entities can be represented by a decision tree:

Lemma 8 (Decision trees represent sparse models). *Let $E \subseteq \mathbf{ent}(X)$ be a subset of entities over X . Then, it is possible to build a boolean decision tree T such that $T(e) = 1$ if and only if $e \in E$. The construction takes time $O(|E||X|)$, and the resulting decision tree has size $O(|E||X|)$.*

Proof. We build the tree iteratively, assuming that we start from the tree that rejects all entities, i.e. the one with only one node (the root) with label 0. Let $E = \{e_1, \dots, e_k\}$, and assume that T_i , for some $0 \leq i < k$, is a tree that accepts all entities $e_1, \dots, e_i$ and rejects any other one. Then, given e_{i+1} we can follow the path defined in T for e_{i+1} and we will reach a leaf with label 0. Then, we extend this tree by appending to this leaf a model that accepts only e_{i+1} (which can be straightforwardly built as a path of length $2|X|$). The resulting tree T_{i+1} only accepts $e_1, \dots, e_{i+1}$, and its size is bounded by $|T_i| + 2|X|$. $\square$

Given a CNF formula φ and an entity e it is possible to simplify φ considering only those atoms that coincide with the assignment dictated by e . This new CNF formula is useful for characterizing the set of sufficient reasons for e with respect to φ .

Definition 9 (CNF formula restricted to an entity). *Let $\varphi = \bigwedge_{i=1}^m C_i$ be a CNF formula where each C_i is a clause over X . For any entity $e \in \mathbf{ent}(X)$ and clause $C = \bigvee_{j=1}^m l_j$ we define:*

$$C^e = \bigvee_{\substack{j=1 \\ l_j(e)=1}}^k l_j$$

Then, we define the model M restricted to e as $M_e = \bigwedge_{i=1}^m C_i^e$.

More precisely, the sufficient reasons for the prediction $\varphi(e)$ will exactly coincide with the set of minimal hitting sets of a certain hypergraph built from φ_e . This is well-known (Liffiton & Sakallah, 2008; Reiter, 1987) and was used in Audemard et al., 2021 to develop an algorithm for computing the relevant and necessary features for predictions of boolean binary decision trees. In Huang, Izza, Ignatiev, and Marques-Silva, 2021 this idea was further extended for a more general class of classifiers, observing that $\mathbf{cnf}(T)_e$ is the disjunction of all contrastive explanations (Marques-Silva, 2023) for prediction $T(e)$.

An hypergraph is a tuple $H = (V, E)$ where V is a set of nodes and $E \subseteq \mathcal{P}(V)$ is a set of hyperedges. We define the size of an hypergraph as $|H| = |V| + \sum_{B \in E} |B|$, and the degree of a node $v \in V$ as $\deg(v) = |\{B \in E : v \in B\}|$. A hitting set of H is a subset $S \subseteq V$ of its nodes such that for any $B \in E$ it is the case that $S \cap B \neq \emptyset$. A hitting set S is minimal if for all $S' \subset S$ it holds that S' is not a hitting set.

3. Auxiliary results

3.1 Hitting sets

We recall the result from Liffiton and Sakallah, 2008² and state it in our terms:

Lemma 10 (Sufficient reasons as Hitting Sets). *Let $M \equiv \bigwedge_{i=1}^m C_i$ be a boolean model over X given as a CNF formula, where each C_i is not tautologically true, and let $e \in \mathbf{ent}(X)$ be an entity such that $M(e) = 1$.*

For any clause C , we denote as $\mathbf{var}(C)$ the set of features from X that appear in the clause C . We can define a hypergraph $H = (X, E)$ using X as the set of nodes; and as hyperedges,

$$E = \{\mathbf{var}(C_i^e) : 1 \leq i \leq m\}$$

Then, it holds that $S \in \mathbf{SR}(M, e)$ if and only if S is a minimal hitting set of H . Also, $|H| = |X| + \sum_{i=1}^m |C_i^e|$.

2. Which actually is Theorem 4.5 from Birnbaum and Lozinskii, 2003

Proof. Let $S \subseteq X$. If $S \cap \text{var}(C_i^e) = \emptyset$ for some i , let e_i be an entity such that $C_i^e(e_i) = 0$ (which exists because of our assumption that the clauses were not tautologically true), and consider the entity e' defined as

$$e'(x) = \begin{cases} e(x) & x \in X \setminus \text{var}(C_i^e) \\ e_i(x) & \text{otherwise} \end{cases}$$

Since $C_i^e(e') = 0$ it holds that $M(e') = 0$. Hence, $S \notin SR(M, e)$.

On the other hand, let's see that if S is a hitting set of H then it must be a reason for e with respect to model M . Let e' be an entity that agrees with e on every feature in S . Then, given any clause C_i^e there is some feature $x \in S \cap C_i^e$ and thus there is some literal in C_i^e that is satisfied by e' . This implies that $C_i^e(e') = 1$, and because this argument works for any i , it must be the case that $M(e') = 1$. □

Moreover, it holds that given any hypergraph H one can build a boolean decision tree T and an entity e such that the minimal hitting sets of H are precisely the sufficient reasons for e with respect to model T .

Lemma 11 (From hitting sets to decision trees). *Let $H = (V, E)$ be a hypergraph. Then, it is possible to build a boolean binary decision tree T over binary features V and an entity e over V such that the set of minimal hitting sets of H is precisely the set of sufficient reasons $SR(T, e)$.*

Moreover, this construction can be done in $O(|V|^2)$ time and the resulting tree has size $O(|V|^2)$.

Proof. Fix the entity e as $e(v) = 1$ for all $v \in V$. For any $B \in E$ we define a clause $C_B^+ \equiv \bigvee_{v \in B} v \in \{1\}$ and likewise a clause $C_B^- \equiv \bigvee_{v \in V \setminus B} v \notin \{1\}$. Consider the model

$$M = \bigwedge_{B \in E} (C_B^+ \vee C_B^-)$$

Note that each clause $C_B^+ \vee C_B^-$ is satisfied by all entities but one. Therefore, M rejects only $|E|$ entities. Then, M can be represented by a decision tree using Lemma 8³. The constructed tree T satisfies $\text{cnf}(T) = \bigwedge_{B \in E} (C_B^+ \vee C_B^-)$, and $\text{cnf}(T)_e = \bigwedge_{B \in E} (C_B^+ \vee C_B^-)^e = \bigwedge_{B \in E} C_B^+$. By Lemma 10 it holds that the sufficient reasons for e with respect to M coincide with the minimal hitting sets of the hypergraph $H' = (V, \{\text{var}(C_B^+) : B \in E\})$, but since $\text{var}(C_B^+) = B$ it is the case that $H' = H$. □

The connection between the sufficient reasons of a decision tree T and the hitting sets of the hypergraph induced by T will be useful for proving both upper and lower bounds. Regarding the latter ones, we provide two simple algorithms for computing minimal hitting sets of hypergraphs.

3. More precisely, one can use Lemma 8 to construct $\neg M$, and then complement all the leaves to get a decision tree representing M .

Lemma 12. *Given a hypergraph $H = (V, E)$ and a subset $W \subseteq V$ of its nodes it is possible to decide if there is a minimal hitting set containing W in $O(|H| \prod_{w \in W} \deg(w))$.*

Proof. The algorithm follows from this simple observation: if S is a minimal hitting set and $v \in S$, then there must be some $B \in E$ such that $S \cap B = \{v\}$. Note that otherwise $S \setminus \{v\}$ would also be a hitting set, contradicting the minimality of S . Thus, if S is a hitting set such that $W \subseteq S$ it must be the case that for each $w \in W$ there is some $B_w \in E$ with $S \cap B_w = \{w\}$.

The algorithm consists of exhaustive search considering all possibilities for the choice of $\{B_w\}_{w \in W} \subseteq E$, which are at most $\prod_{w \in W} \deg(w)$. Given any choice of $\{B_w\}_{w \in W}$ we need to decide if there is a hitting set included in $V \setminus (\bigcup_{w \in W} (B_w \setminus \{w\}))$, which is the same as checking if $B \not\subseteq \bigcup_{w \in W} (B_w \setminus \{w\})$ for all $B \in E$. If this holds, then there is a minimal hitting set of H contained in $V \setminus \bigcup_{w \in W} B_w \setminus \{w\}$, and this minimal hitting set must contain each w , since otherwise the hyperedge B_w would not be covered.

Given a choice of $\{B_w\}_{w \in W}$ we can decide in $O(|H|)$ time whether $V \setminus (\bigcup_{w \in W} (B_w \setminus \{w\}))$ is a hitting set of H . Thus, the final complexity is $O(|H| \prod_{w \in W} \deg(w))$. $\square$

Lemma 13. *Given a hypergraph $H = (V, E)$ and a node $v \in V$ it is possible to decide if there are k different minimal hitting sets containing v in $O(k|H|(\deg(v)|V|^{k-1} + |V|))$.*

Proof. Assuming we have computed $k-1$ minimal hitting sets $S_1, \dots, S_{k-1}$, we show how to compute another one, namely S_k . Using the same observation from Lemma 12 we conclude that if S is a minimal hitting set containing v , then there must be some hyperedge $B \in E$ such that $S \cap B = \{v\}$. Moreover, if we want a hitting set S_k such that $S_i \neq S_k$ then there must be some node $s_i \in S_i$ such that $s_i \notin S_k$, and this holds for each $1 \leq i \leq k-1$.

This reasoning implies an exhaustive search algorithm analogous to the one from Lemma 12. If there is another minimal hitting set S_k that contains v then there must be some $B \in E$ with $v \in B$ and nodes $s_1, \dots, s_{k-1}$ with $s_i \in S_i$ such that $S_k \subseteq V \setminus ((B \setminus \{v\}) \cup \{s_1, \dots, s_{k-1}\})$. Therefore, for each choice of B (of which there are $\deg(v)$) and of the nodes $s_1, \dots, s_{k-1}$ (which are at most $|V|^{k-1}$) we can check whether $V \setminus ((B \setminus \{v\}) \cup \{s_1, \dots, s_{k-1}\})$ is a hitting set, and if it is then we can obtain S_k by iteratively removing nodes until achieving minimality. This last procedure can be implemented in time $O(|H||V|)$, and thus we can find an S_k if it exists in time $O(|H| \deg(v)|V|^{k-1} + |H||V|)$.

To compute all the minimal hitting sets we need to invoke the previous algorithm k times, and thus we can bound the whole complexity by $O(k|H|(\deg(v)|V|^{k-1} + |V|))$. $\square$

The algorithm from Lemma 12 is not polynomial with respect to the input size, and this situation cannot be improved in general:

Proposition 14. *The problem of deciding, given a graph $G = (V, E)$ and a subset of nodes $W \subseteq V$, whether there is a minimal vertex cover⁴ S of G such that $W \subseteq S$ is NP-COMPLETE.*

Proof. The NP membership is immediate. For the hardness, we show a reduction from 3-COLORING to our problem. Nonetheless, for clarity we actually provide a reduction of

4. In the context of graphs a hitting set is a vertex cover.

3-COLORING to another problem we named HITTING-SET-FP, and then a reduction from HITTING-SET-FP to our problem.

HITTING-SET-FP is the problem of finding a hitting set of a hypergraph when some pairs of nodes are forbidden from being picked together. That is, the problem consists of deciding, given a hypergraph $H = (V, E)$ and some forbidden pairs $F \subseteq V \times V$, whether there is a hitting set S of H such that for every pair $(v_1, v_2) \in F$ it is the case that $\{v_1, v_2\} \not\subseteq S$. It is clear that this problem is in NP.

We now provide the reduction from 3-COLORING to HITTING-SET-FP. Given a graph $G = (V_G, E_G)$ we define a hypergraph $H = (V_H, E_H)$ as

$$\begin{aligned} V_H &= \{v_i : v \in V_G, 1 \leq i \leq 3\} \\ E_H &= \{\{v_1, v_2, v_3\} : v \in V_G\} \end{aligned}$$

with forbidden pairs $F = \{(v_i, w_i) : vw \in E_G, 1 \leq i \leq 3\} \cup \{(v_i, v_j) : v \in V_G, 1 \leq i < j \leq 3\}$.

Intuitively, each node of H corresponds to an assignment of a node from G to a color. The set of hyperedges enforces that one color must be chosen for each node, and the set of forbidden pairs enforces that nodes joined by an edge must use different colors and that each node must have a unique color. The correctness of the reduction is immediate.

For the second reduction (from HITTING-SET-FP to our problem), consider a hypergraph $H = (V_H, E_H)$ with forbidden pairs F . We define a graph $G = (V_G, E_G)$ as

$$\begin{aligned} V_G &= V_H \cup E_H \\ E_G &= \{vw : (v, w) \in F\} \cup \{(v, B) : B \in E_H, v \in B\} \end{aligned}$$

G has all the nodes H has, and one extra node for each hyperedge. Edges are put between the two nodes of every forbidden pair and between any node from V_H and the hyperedges to which it belongs.

We now show that V_H has a hitting set that does not contain any forbidden pair from F if and only if V_G has a minimal hitting set containing E_H . This implies the correctness of the reduction by picking $W = E_H$.

Let S be a valid hitting set of H . Then each edge $vB \in E_G$ is covered by $V_G \setminus S$ because $B \in E \subseteq V_G \setminus S$. Similarly, for every $(v, w) \in F$ it is the case that either v or w does not belong to S , and thus one of them belongs to $V_G \setminus S$, which implies that the edge vw from G is covered. Therefore, since $V_G \setminus S$ is a vertex cover there is some minimal vertex cover $C \subseteq V_G \setminus S$. We show that $B \in C$ for any $B \in E$: because S is a hitting set, it holds that there is some $s_B \in S \cap B$, and thus if it were the case that $B \notin C$, then the edge $s_B B$ would be left uncovered. We conclude that there is a minimal vertex cover of V_G containing E_H .

For the other direction, let C be a minimal vertex cover of V_G such that $E_H \subseteq C$. Then, we claim that $V_H \setminus C$ is a valid hitting set of H . For each $B \in E$ it holds that each edge vB is covered in V_G , for every $v \in B$. Since the covering is minimal, there must be some node $s_B \in B$ such that $s_B \notin C$, and thus $s_B \in V_H \setminus C$. This implies that $V_H \setminus C$ is a hitting set. To see that it is valid, consider any forbidden pair (v, w) : since vw is an edge from V_G it must be the case that either $v \in C$ or $w \in C$, and thus either $v \notin V_H \setminus C$ or $w \notin V_H \setminus C$. We conclude that $V_H \setminus C$ is a valid hitting set. $\square$

We presented Lemmas 12 and 13 merely to show that these problems, which are intractable in general (i.e. when either $|W|$ or k is not fixed), may allow for efficient solutions in restricted cases. In case one needed to enumerate hitting sets efficiently these algorithms will not be optimal, and should rather resort to more efficient alternatives (Gainer-Dewar & Vera-Licona, 2017).

4. Main results

4.1 Relevant features

Consider the following problem:

PROBLEM: $\text{RELEVANT}_{\mathcal{M}}$
 INPUT: A model $M \in \mathcal{M}$ over X , an entity $e \in \text{ent}(X)$ and a feature $x \in X$.
 OUTPUT: Is x relevant for the prediction $M(e)$?

In Audemard et al., 2021 this problem was shown to be tractable for $\mathcal{M} = \text{DT}_B$. Huang, Izza, Ignatiev, and Marques-Silva, 2021 extended this result for a more general class of trees (in our terms, trees with only categorical features) and later Huang et al., 2023 showed that it is NP-COMPLETE for FBDDs. We show that the algorithm from Audemard et al., 2021 can be extended to our more general version of decision trees, without affecting its complexity⁵:

Theorem 15. *The problem $\text{RELEVANT}_{\text{DT}}$ can be solved in time $O(|X||T|^2)$.*

Proof. We describe the algorithm to solve the problem. Given the decision tree T and the entity e , compute $k = T(e)$. Now, relabel all leaves with label k to 1, and the rest to 0. The obtained tree T_B is now a boolean decision tree such that $T_B(e) = 1$. It can be easily shown that $SR(T, e) = SR(T_B, e)$.

Using Lemma 10 we can construct a hypergraph H from $\text{cnf}(T_B)^e$ such that x is relevant for $T_B(e)$ if and only if x belongs to a minimal hitting set of H . We can decide this using the algorithm from Lemma 12 in time $O(|H| \deg(v)) = O(|X||T|^2)$. $\square$

As explained in Audemard et al., 2021, the previous algorithm can actually be used to compute all relevant features in time $O(|X||T|^2)$ by removing from H all hyperedges B that contain another hyperedge. In the resulting hypergraph any node belonging to some hyperedge must belong to a minimal hitting set, and thus be a relevant feature.

Many features can be relevant, and as a consequence relevancy is not a notion strong enough to rank features properly. We now propose two problems that generalize the notion of relevancy, show that are NP-COMPLETE in general, but that they can be solved efficiently for restricted instances.

Proposition 16. *The problem of counting, given a boolean binary decision tree T , an entity e and a feature x , the number of sufficient reasons for $T(e)$ containing x , is #P-COMPLETE.*

5. The complexity stated in Proposition 15 actually does not agree with the one stated in Audemard et al., 2021[Proposition 6] because we believe the proper bound for their algorithm is $O(|X||T|^2)$.

Also, deciding if there are at least k sufficient reasons for $T(e)$ containing x can be done in time $O(k|X|^{k+2}|T|)$.

Proof. The hardness results follows by a reduction from the problem of counting the minimal vertex covers of a graph, which is #P-COMplete (Okamoto et al., 2005)[Theorem 8].

Let $G = (V, E)$ be a graph, and consider $G' = (V \cup \{v, w\}, E \cup \{vw\})$ where $v, w \notin V$. Then, it holds that the number of minimal vertex covers of G coincides exactly with the number of minimal vertex covers of G' containing node v . Thus, using Lemma 11 we can build a binary boolean decision tree $T_{G'}$ and an entity e such that the number of minimal vertex covers of G' containing v coincides exactly with the number of sufficient reasons of $T_{G'}(e)$ containing v .

Regarding the algorithm, it follows immediately using Lemmas 10 and 13. $\square$

Proposition 17. *The problem of deciding, given a decision tree T , an entity e and a subset of features $Y \subseteq X$, whether there is a sufficient reason $S \in SR(T, e)$ such that $Y \subseteq S$, is NP-COMplete. Also, it can be solved in time $O(|X||T|^{|Y|+1})$.*

Proof. The NP membership is immediate. For the hardness, by Lemma 11 we can reduce the problem of computing a minimal hitting set containing some set of nodes (which is NP-HARD by Lemma 13) to our problem.

For the algorithm, apply Lemmas 10 and 12. $\square$

Even though these problems turn out to be intractable in general, the algorithmic results from Propositions 16 and 17 imply that there exist efficient algorithms generalizing slightly the notion of relevancy in decision trees. Meanwhile, for models that are more expressive than FBDDs this is known to be impossible because of the intractability result for RELEVANT_{FBDD}. We observe that the case for OBDDs remains open.

4.2 Necessary features

Consider the following problem:

PROBLEM: NECESSARY _{$\mathcal{M}$}
 INPUT: A model $M \in \mathcal{M}$ over X , an entity $e \in \mathbf{ent}(X)$ and a feature $x \in X$.
 OUTPUT: Is x necessary for the prediction $M(e)$?

It was proven in Audemard et al., 2021 that this problem can be solved in time $O(|X||M|^2)$ for $\mathcal{M} = \text{DT}_B$. Furthermore, in Huang et al., 2023 it was shown that the problem is tractable for any class of models for which the problem of deciding if a subset of features is a reason (not requiring minimality) is tractable. This implies tractability of the problem for FBDDs and many other families of circuits such as D-DNNFs. We extend this result using the following characterization, which highlights the fact that the condition of being necessary is strong:

Theorem 18 (Characterization of necessary features). *Let $\mathcal{M}$ be a model over features X and $e \in \mathbf{ent}(X)$. Then, a feature $x \in X$ is necessary for $M(e)$ if and only if $M(e) \neq M(e_{x=b})$ for some $b \in \mathcal{D}_x$.*

Proof. Observe that x is necessary for $M(e)$ if and only if $x \in S$ for every $S \in SR(M, e)$, which happens if and only if $X \setminus \{x\}$ is not a reason for $M(e)$ (since otherwise there would be some sufficient reason $S \subseteq X \setminus \{x\}$)⁶. It holds that $X \setminus \{x\}$ is a reason if and only if $M(e) = M(e_{x=b})$ for every $b \in \mathcal{D}_x$. $\square$

Thus, it is easy to compute necessary features for any model class that can be evaluated efficiently as long as $|\mathcal{D}_x|$ is small.

Corollary 19. *The problem $\text{NECESSARY}_{\mathcal{M}}$ can be solved in time $O(\text{eval}(M)|\mathcal{D}_x|)$, where $\text{eval}(M)$ denotes the time complexity required to evaluate the model M .*

In particular this implies that the problem is tractable for the class of boolean binary classifiers (which contains the class of DNFs and general binary decision diagrams).

Corollary 19 implies that the necessary features for a prediction can be computed efficiently for any model as long as the domain of the features is bounded.

When considering numerical features Corollary 19 is useless. Nonetheless, we observe that for “comparison-based models” (i.e. models that only query values of the entity through comparisons of the form $e(x) \leq b$) the non-finite (and even non-discrete) domain of numerical features can be *discretized* using well-known ideas (Darwiche & Ji, 2022). For example, in the case of our version of decision trees, it suffices to look for all nodes querying the value of feature x as $x \leq t_1, \dots, x \leq t_m$ with $t_1 \leq t_2 \leq \dots \leq t_m$ and then consider the finite classes induced by the intervals $[m_x, t_1], (t_1, t_2], \dots, (t_{m-1}, t_m], (t_m, M_x]$. Such a strategy would also work for any graph-based classifier à la Huang, Izza, Ignatiev, and Marques-Silva, 2021. Note that the number of classes that have to be considered grows linearly with the number of nodes, and thus we conclude tractability of the problem.

Nonetheless, for our class DT we can compute all necessary features in linear time.

Proposition 20. *Given a decision tree T over features X and $e \in \text{ent}(X)$ it is possible to compute all the necessary features for $T(e)$ in $O(|X| + |T|)$.*

Proof. We assume that T is a boolean decision tree and that $T(e) = 1$, which can be done without loss of generality using the techniques employed for the algorithm from Theorem 15.

We will need the primitive $\text{taut}_T(v, e, x, (a, b])$, which: given a node v from a boolean decision tree T , an entity e , a numerical feature x and a range $(a, b]$, decides whether $T_v(e_{x=c}) = 1$ for all $c \in (a, b]$. It can be implemented in linear time using recursion as detailed in Figure 3. We also need an analogous version $\text{taut}_T(T, e, x, Z)$ where x is a categorical feature and $Z \subseteq \mathcal{D}_x$, which we detail in Algorithm 4.

Given these functions, it holds that a numerical feature x is necessary for $T(e)$ if and only if $\text{taut}_T(r, e, x, \mathcal{D}_x) = 1$ where r is the root of T . Thus, we can find each necessary feature in $O(|T|)$. In order to achieve a linear time algorithm to compute all the necessary features we improve on this idea.

Let P be the path traversed from the root to some leaf when processing entity e , and let $v_0, \dots, v_k$ be the nodes of P where a comparison against x is made, naming $l(v_i) \equiv x \leq t_i$. We define a sequence of intervals $I_0, \dots, I_k, I_{k+1}$ that partition $(m_x, M_x]$ as follows⁷:

6. This was proven in Huang et al., 2023[Proposition 2].

7. We assume that the interval is open on the left side for simplicity.

Algorithm 1 $\text{taut}_T(v, e, x, (a, b])$

Require: v is a node from T , e an entity for T and x a numerical feature

```

1: if  $(a, b] = \emptyset$  then
2:   return 1
3: else if  $v$  is a leaf with label  $b$  then
4:   return  $b$ 
5: end if
6:  $w_1 \leftarrow$  left child of  $v$ 
7:  $w_2 \leftarrow$  right child of  $v$ 
8: if  $\text{feat}(v) \neq x \wedge l(v)(e) = 1$  then
9:   return  $\text{taut}_T(w_1, e, x, (a, b])$ 
10: else if  $\text{feat}(v) \neq x \wedge l(v)(e) = 0$  then
11:   return  $\text{taut}_T(w_2, e, x, (a, b])$ 
12: else if  $\text{feat}(v) = x \wedge \text{value}(v) \in (a, b)$  then
13:   return  $\text{taut}_T(w_1, e, x, (a, \min\{b, \text{value}(v)\}]) \wedge \text{taut}_T(w_2, e, x, (\max\{a, \text{value}(v)\}, b])$ 
14: end if

```

Figure 3: Algorithm to decide whether $T_v(x_{x=c}) = 1$ for all $c \in (a, b]$

Algorithm 2 $\text{taut}_T(v, e, x, Z)$

Require: v is a node from T , e an entity for T and x a categorical feature

```

1: if  $Z = \emptyset$  then
2:   return 1
3: else if  $v$  is a leaf with label  $b$  then
4:   return  $b$ 
5: end if
6:  $w_1 \leftarrow$  left child of  $v$ 
7:  $w_2 \leftarrow$  right child of  $v$ 
8: if  $\text{feat}(v) \neq x \wedge l(v)(e) = 1$  then
9:   return  $\text{taut}_T(w_1, e, x, Z)$ 
10: else if  $\text{feat}(v) \neq x \wedge l(v)(e) = 0$  then
11:   return  $\text{taut}_T(w_2, e, x, Z)$ 
12: else if  $\text{feat}(v) = x \wedge \text{value}(v) \in (a, b)$  then
13:   return  $\text{taut}_T(w_1, e, x, Z \cap \text{values}(v)) \wedge \text{taut}_T(w_2, e, x, Z \cap (\mathcal{D}_x \setminus \text{values}(v)))$ 
14: end if

```

Figure 4: Algorithm to decide whether $T_v(e_{x=b}) = 1$ for all $b \in Z$.

1. If $l(v_0)(e) = 1$, let $I_0 = (t_0, M_x]$. Otherwise, let $I_0 = (m_x, t_0]$.
2. For $1 \leq i \leq k$, if $l(v_i)(e) = 1$ then $I_i = ([m_x, M_x] \setminus \cup_{j=0}^{i-1} I_j) \cap (t_i, M_x]$, and otherwise $I_i = ([m_x, M_x] \setminus \cup_{j=0}^{i-1} I_j) \cap (m_x, t_i]$.
3. I_{k+1} is equal to $(m_x, M_x] \setminus \cup_{i=0}^k I_i$.

Note that $I_0 \dots I_{k+1}$ are indeed intervals of the form $(a, b]$ because $I_i = \bigcap_{j=0}^{i-1} I_j^c \cap (a_i, b_i]$ where $I_j^c = (m_x, M_x] \setminus I_j$ and $(a_i, b_i]$ is an interval that depends on the value of $l(v_i)(e)$. I_i denotes the interval of values for x such that any entity of the form $e_{x=c}$ for $c \in I_i$ will reach node v_i when being processed by e and then diverge from the path P .

We claim that $\mathbf{taut}_T(r, e, x, (m_x, M_x]) = \bigwedge_{i=0}^k \mathbf{taut}_T(v_i, e, x, I_i)$. To prove this, it is enough to show that $\mathbf{taut}_T(r, e, x, I_i) = \mathbf{taut}_T(v_i, e, x, I_i)$, considering that the intervals partition the range of values for feature x . The equality holds because of the definition of the intervals and our previous remarks. We note that there is no need to consider the interval I_{k+1} because those entities whose value for x is in this range I_{k+1} will follow path P until the end, and thus will be evaluated to 1.

For the categorical features an analogous argument can be made, and thus the call to $\mathbf{taut}_T(r, e, x, \mathcal{D}_x)$ can be decomposed as $\bigwedge_{i=0}^k \mathbf{taut}_T(v_i, e, x, Z_k)$ where the sets $\{Z_i\}_{0 \leq i \leq k+1}$ partition $\mathcal{D}_x$.

With this decomposition we can now obtain all necessary features in linear time. The algorithm consists in traversing P , keeping for each feature the current interval (in the case of numerical features) or subset of values (for the categorical ones) consistent with all entities reaching the current node. Then, at every such node we call $\mathbf{taut}_T$ according to the feature being considered in the node and the current set of consistent values. Note that each call to $\mathbf{taut}_T$ will process different nodes, and thus the whole complexity is $O(|X| + |T|)$. $\square$

Using similar ideas one can compute all the necessary features for any FBDD model.

Proposition 21. *Given an FBDD D over features X and $e \in \mathbf{ent}(X)$ it is possible to compute all necessary features for $M(e)$ in $O(|X| + |D|)$.*

Proof. We use the same ideas from Proposition 20. Let D be an FBDD and e some entity. Through dynamic programming we can compute, for each node v in D , the value $D_v(e)$ in $O(|D|)$ time observing that the following relation holds:

$$D_v(e) = \begin{cases} b & v \text{ is a leaf with label } b \\ D_{\mathbf{left}(v)}(e) & e(\mathbf{feat}(v)) = 0 \\ D_{\mathbf{right}(v)}(e) & e(\mathbf{feat}(v)) = 1 \end{cases}$$

Let P be the path followed in D when processing e . Note that a feature x is necessary if and only if $x = \mathbf{feat}(v)$ for some $v \in P$ and $D_{\mathbf{left}(v)} = D_{\mathbf{right}(v)} = D(e)$. To see this, note that there is a unique node $v \in P$ such that $x = \mathbf{feat}(v)$ because of the read-once property of FBDDs. Also, nodes reachable from v will not use the feature x again, and thus $D(e_{x=0}) = D_{\mathbf{left}(v)}(e)$ and $D(e_{x=1}) = D_{\mathbf{right}(v)}(e)$.

Therefore, after the precomputation step we can traverse the path P and decide in $O(1)$ for each feature whether it is necessary. $\square$

Naturally, the previous algorithm can be used in OBDDs because they are particular cases of FBDDs. We note that it cannot be straightforwardly generalized to arbitrary diagrams because we required the read-once property for the correctness of the feature necessity decision criterion.

As a closing remark, we mention that the characterization from Theorem 18 is bitter-sweet, since it implies that the condition of being necessary is particularly strong, and thus maybe suboptimal regarding explainability tasks.

4.3 Useful features

Consider the following problem:

PROBLEM: $\text{USEFUL}_{\mathcal{M}}$
 INPUT: A model $M \in \mathcal{M}$ over X and a feature $x \in X$.
 OUTPUT: Is x useful for M ?

We can characterize useful features as those that are relevant for some entity and at the same time as those that are necessary for some entity.

Theorem 22 (Characterization of useful features). *Let $\mathcal{M}$ be a model over X and $x \in X$ a feature. Then, the following are equivalent:*

- i) x is useful for $\mathcal{M}$.
- ii) There is some $e \in \text{ent}(X)$ such that x is necessary for $M(e)$.
- iii) There is some $e \in \text{ent}(X)$ such that x is relevant for $M(e)$.

Proof. i) is equivalent to ii) because of Theorem 18. ii) implies iii) because every necessary feature is also relevant.

To prove that iii) implies i) let $e \in \text{ent}(X)$ be an entity such that x is relevant, and let $S \in \text{SR}(M, e)$ be a sufficient reason such that $x \in S$. Then, since S is minimal, $S \setminus \{x\}$ is not a reason, and thus there is some entity $e' \in \text{cw}(e, S \setminus \{x\})$ such that $M(e') \neq M(e)$. Note that since S is a reason it must be the case that $e'(x) \neq e(x)$.

It holds that $M(e'_{x=e(x)}) = M(e) \neq M(e')$, and therefore, we conclude that x is useful. $\square$

Let $\text{EQUIV}_{\mathcal{M}}$ be the problem of deciding, given two models $M_1, M_2 \in \mathcal{M}$, whether $M_1 \equiv M_2$. We observe that this problem is related to $\text{USEFUL}_{\mathcal{M}}$:

Proposition 23. *Let $\mathcal{M}$ be a class of models closed by conditioning. Then, $\overline{\text{USEFUL}_{\mathcal{M}}} \leq_p \text{EQUIV}_{\mathcal{M}}$.*

Let $\mathcal{M}$ be a class of models closed by disjoint disjunction. Then, $\text{EQUIV}_{\mathcal{M}} \leq_p \overline{\text{USEFUL}_{\mathcal{M}}}$.

Proof. For the first part, observe that x is useful for a model M if and only if $M \neq M_{x=b}$ for all $b \in \mathcal{D}_x$. For the second part, note that $M_1 \equiv M_2$ if and only if x is not useful for the model $(M_1 \wedge x) \vee (M_2 \wedge \bar{x})$. $\square$

Observe that $\text{USEFUL}_{\mathcal{M}}$ is in CONP in general because we can decide if x is not useful by guessing an entity e and a value $b \in \mathcal{D}_x$. Together with Proposition 23 this implies the CONP -completeness of the problem $\text{USEFUL}_{\text{DNF}}$. Similarly, it implies tractability for the classes DT and OBDD . Finally, for classes such as FBDD or D-DNNF the complexity remains unknown because the complexity of both $\text{EQUIV}_{\text{FBDD}}$ and $\text{EQUIV}_{\text{D-DNNF}}$ is unknown.

If x is useful then there is some entity for which it is necessary. Moreover, we can assign an “importance score” to a feature x by considering the number of entities for which it is necessary. Note that this makes sense only for models with only categorical features, since otherwise the number of entities could be infinite.

This scoring idea can be related to the one considered in Bertossi et al., 2020 based on the notion of *counterfactual cause* (Halpern & Pearl, 2005). In their context, a counterfactual cause for an entity e on the prediction of a boolean classifier M such that $M(e) = 1$ is a feature x alongside a value b such that $M(e_{x=b}) = 0$. Our proposed score assigns importance to a feature proportional to the number of entities admitting a counterfactual cause based on that feature.

This score can be computed through a reduction to model counting for models closed by conditioning, negation and conjunction. From now on, given a boolean model M we denote by $CT(M)$ the number of entities that M accepts.

Proposition 24. *Let $M \in \mathcal{M}$ be a categorical boolean model whose model class is closed by conditioning, negation and conjunction. Then, the value $|\{e \in \text{ent}(X) : x \text{ is necessary for } M(e)\}|$ can be computed using 2 calls to model counting for class $\mathcal{M}$.*

Proof. We will count the number of entities for which x is not necessary. By Theorem 18 x is not necessary for e if and only if $k = M(e) = M(e_{x=b})$ for all $b \in \mathcal{D}_x$. Thus, the number of entities for which x is not necessary is $CT(\bigwedge_{b \in \mathcal{D}_x} M_{x=b}) + CT(\neg \bigwedge_{b \in \mathcal{D}_x} M_{x=b})$ $\square$

Corollary 25. *Given a boolean decision tree $T \in \text{DT}_B$ and a feature x , it is possible to compute the number of entities for which x is necessary in $O(|T|^2)$.*

Proof. Given two decision trees T_1, T_2 , the model $T_1 \wedge T_2$ can be represented by a tree of size $O(|T_1||T_2|)$ appending to each leaf with label 1 of T_1 a copy of T_2 . Moreover, model counting in trees can be done in linear time (Darwiche & Marquis, 2002). Thus, in this case the algorithm from Proposition 24 can be implemented in time $O(|T|^2)$. $\square$

We remark that Proposition 24 can be generalized for k -class models as long as it is possible to “booleanize” the k -class model M by identifying one class c as 1 and the others as 0. If the classes of M are $\{0, \dots, k-1\}$ and M^c for $0 \leq c < k$ denotes the booleanized version of M where c is identified with 1, then the number of entities for which x is not necessary can be computed as $\sum_{i=0}^{k-1} CT(\bigwedge_{b \in \mathcal{D}_x} M_{x=b}^i)$.

5. Experiments

6. Conclusions

In this work we extended previous results regarding the computation of relevant and necessary features, and also proposed new generalizations of the notion of relevancy. Moreover,

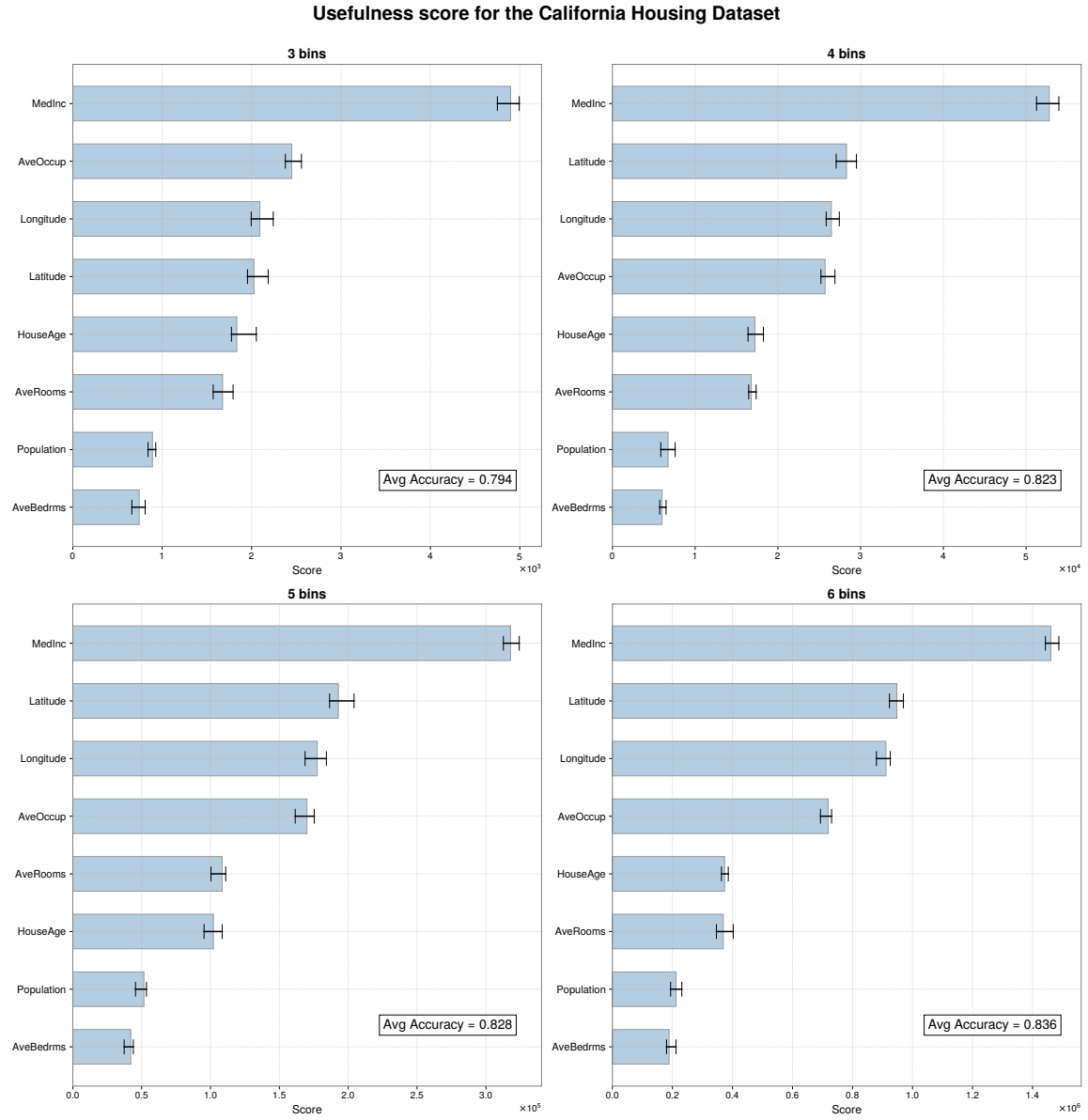


Figure 5: Results for the California Housing Dataset

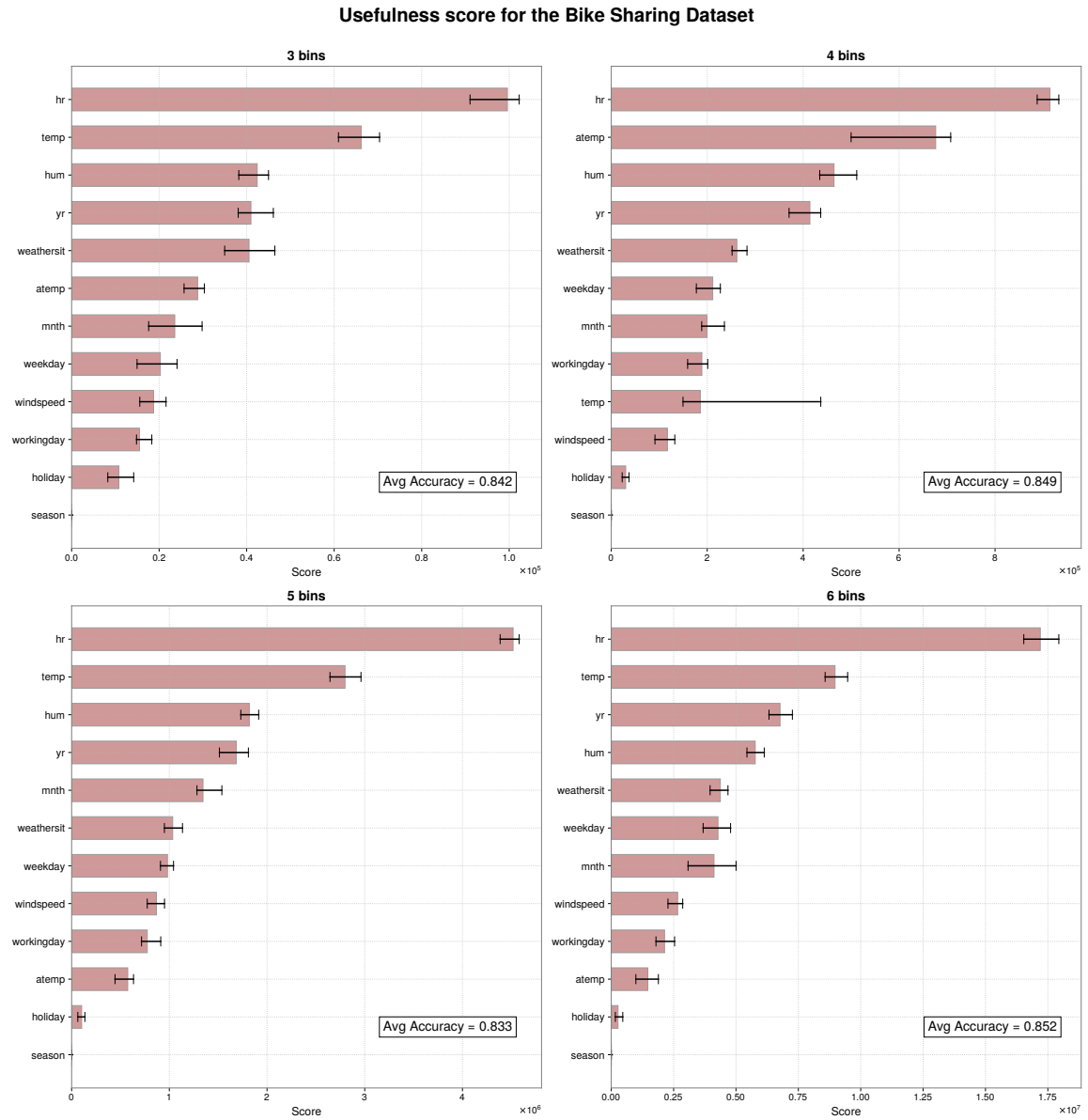


Figure 6: Results for the Bike Sharing Dataset

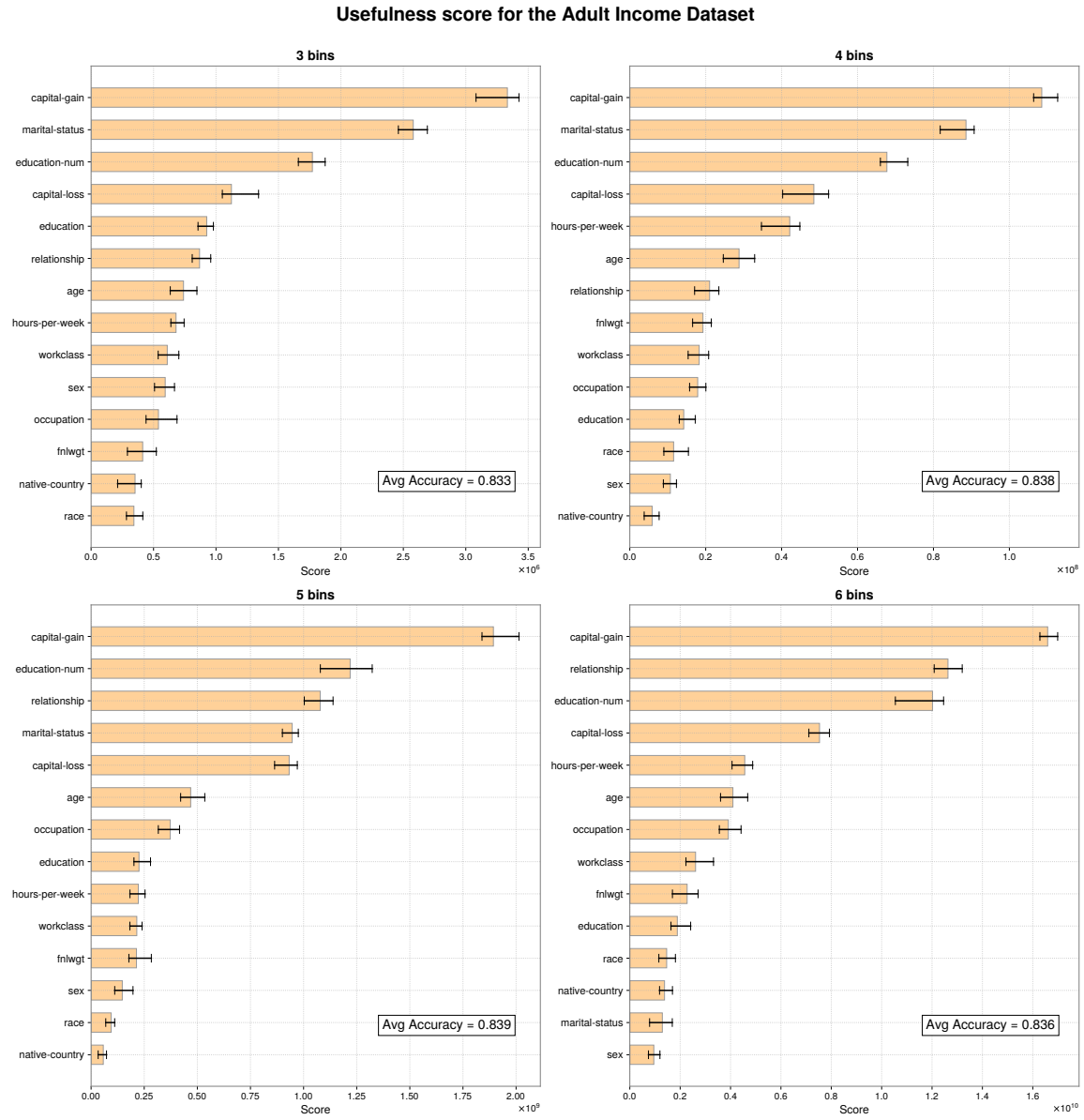


Figure 7: Results for the Adult Income Dataset

we defined a global notion of feature importance, and showed that it is related to the local ones induced by relevancy and necessity. We studied many problems involving these parameters, proving them to be intractable for complex classes of models in some cases (such as detecting *usefulness* in DNFs) but also finding tractable restrictions (for example, detecting *necessity* in DNFs). For the tractable cases we described our algorithms with precision, allowing for an easy implementation. In addition, some of them run in linear time, and thus are highly efficient.

Regarding feature relevancy, we extended the results from Audemard et al., 2021 and Huang et al., 2022 considering more general decision trees, similar to those studied in Darwiche and Ji, 2022. We also proposed two generalizations of the problem of computing relevant features, which we showed to be intractable in general (Propositions 16 and 17) but solvable in the case of decision trees under certain hypotheses. Moreover, to attain the hardness results we proved hardness for a common vertex cover problem which, as far as we know, has not been studied from the point of view of computational complexity before (Lemma 13). We note that the problem of computing relevant features for slightly more expressive models, such as OBDDs, is still open, since Huang, Izza, Ignatiev, and Marques-Silva, 2021 only shows hardness for FBDDs.

On the topic of feature necessity, we extended the results of Huang et al., 2023, showing that computing necessary features is feasible for all reasonable models (i.e. those allowing evaluation in polynomial time) with categorical features (Corollary 19). Furthermore, we provided linear time algorithms for the case of decision trees with categorical and numerical features, and for FBDDs as well (Propositions 20 and 21). We left open the case of extending this linear time algorithms for arbitrary diagrams, or rather to consider restricted families of circuits such as D-DNNFs.

Finally, we introduced the notion of feature *usefulness*, and showed how it is related to relevancy and necessity (Theorem 22). We showed that the problem of computing useful features is related to the problem of detecting whether two models are equivalent (Proposition 23), and in particular this implied intractability for expressive models such as DNFs and tractability for simple ones such as decision trees and OBDDs. The case for FBDDs and D-DNNFs remains open. We also considered measuring usefulness as a counting problem, and provided general algorithms that were efficient in the particular case of decision trees (Proposition 24 and Corollary 25).

We note that in this work we did not perform any experiments to verify whether the score induced by our notion of usefulness is helpful in the process of explaining a model's behaviour. Thus, this is left as future work.

References

- Angwin, J., Larson, J., Mattu, S., & Kirchner, L. (2022). Machine bias. In *Ethics of data and analytics* (pp. 254–264). Auerbach Publications.
- Arenas, M., Barceló, P., Bertossi, L., & Monet, M. (2023). On the complexity of SHAP-score-based explanations: Tractability via knowledge compilation and non-approximability results. *Journal of Machine Learning Research*, 24(63), 1–58.
- Audemard, G., Bellart, S., Bounia, L., Koriche, F., Lagniez, J.-M., & Marquis, P. (2021). On the explanatory power of decision trees. *arXiv preprint arXiv:2108.05266*.

- Audemard, G., Koriche, F., & Marquis, P. (2020). On tractable XAI queries based on compiled representations. *17th International Conference on Principles of Knowledge Representation and Reasoning (KR'20)*.
- Audemard, G., Lagniez, J.-M., Marquis, P., & Szczepanski, N. (2023). Computing abductive explanations for boosted trees. *International Conference on Artificial Intelligence and Statistics*, 4699–4711.
- Barceló, P., Monet, M., Pérez, J., & Subercaseaux, B. (2020). Model interpretability through the lens of computational complexity. *Advances in neural information processing systems*, 33, 15487–15498.
- Bertossi, L., & León, J. E. (2023). Compiling neural network classifiers into boolean circuits for efficient SHAP-score computation. *Proceedings of AMW*.
- Bertossi, L., Li, J., Schleich, M., Suciu, D., & Vagena, Z. (2020). Causality-based explanation of classification outcomes. *Proceedings of the Fourth International Workshop on Data Management for End-to-End Machine Learning*, 1–10.
- Birnbaum, E., & Lozinskii, E. L. (2003). Consistent subsets of inconsistent systems: structure and behaviour. *Journal of Experimental & Theoretical Artificial Intelligence*, 15(1), 25–46.
- Brynjolfsson, E., & McAfee, A. (2014). *The second machine age: Work, progress, and prosperity in a time of brilliant technologies*. WW Norton & company.
- Brynjolfsson, E., & Mitchell, T. (2017). What can machine learning do? Workforce implications. *Science*, 358(6370), 1530–1534.
- Carbannel, C., Cooper, M., & Marques-Silva, J. (2023). Tractable explaining of multivariate decision trees. *KR 2023-20th International Conference on Principles of Knowledge Representation and Reasoning*, 127–135.
- Darwiche, A., et al. (2004). New advances in compiling CNF to decomposable negation normal form. *Proc. of ECAI*, 328–332.
- Darwiche, A., & Ji, C. (2022). On the computation of necessary and sufficient explanations. *Proceedings of the AAAI Conference on Artificial Intelligence*, 36, 5582–5591.
- Darwiche, A., & Marquis, P. (2002). A knowledge compilation map. *Journal of Artificial Intelligence Research*, 17, 229–264.
- De Colnet, A., & Mengel, S. (2020). Lower bounds for approximate knowledge compilation. *arXiv preprint arXiv:2011.13721*.
- European Commission. (2018). Coordinated plan on Artificial Intelligence. *COM (2018) 795 Final*.
- Fryer, D., Strümke, I., & Nguyen, H. (2021). Shapley values for feature selection: The good, the bad, and the axioms. *IEEE Access*, 9, 144352–144360.
- Gainer-Dewar, A., & Vera-Licona, P. (2017). The minimal hitting set generation problem: algorithms and computation. *SIAM Journal on Discrete Mathematics*, 31(1), 63–100.
- Goodman, B., & Flaxman, S. (2017). European Union regulations on algorithmic decision-making and a “right to explanation”. *AI magazine*, 38(3), 50–57.
- Gunning, D., & Aha, D. (2019). DARPA’s Explainable Artificial Intelligence (XAI) Program. *AI Magazine*, 40, 44–58.
- Halpern, J. Y., & Pearl, J. (2005). Causes and explanations: A structural-model approach. Part I: Causes. *The British journal for the philosophy of science*.
- Huang, X., Cooper, M. C., Morgado, A., Planes, J., & Marques-Silva, J. (2023). Feature necessity & relevancy in ML classifier explanations. *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, 167–186.
- Huang, X., Izza, Y., Ignatiev, A., Cooper, M., Asher, N., & Marques-Silva, J. (2022). Tractable explanations for d-DNNF classifiers. *Proceedings of the AAAI Conference on Artificial Intelligence*, 36, 5719–5728.
- Huang, X., Izza, Y., Ignatiev, A., Cooper, M. C., Asher, N., & Marques-Silva, J. (2021). Efficient explanations for knowledge compilation languages. *arXiv preprint arXiv:2107.01654*.

- Huang, X., Izza, Y., Ignatiev, A., & Marques-Silva, J. (2021). On efficiently explaining graph-based classifiers. *arXiv preprint arXiv:2106.01350*.
- Huang, X., & Marques-Silva, J. (2023). The inadequacy of Shapley values for explainability. *arXiv preprint arXiv:2302.08160*.
- Izza, Y., Ignatiev, A., & Marques-Silva, J. (2020). On explaining decision trees. *arXiv preprint arXiv:2010.11034*.
- Liffiton, M. H., & Sakallah, K. A. (2008). Algorithms for computing minimal unsatisfiable subsets of constraints. *Journal of Automated Reasoning*, 40, 1–33.
- Lu, Y. (2019). Artificial intelligence: a survey on evolution, models, applications and future trends. *Journal of Management Analytics*, 6(1), 1–29.
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in neural information processing systems*, 30.
- Madiega, T. A. (2019). EU guidelines on ethics in artificial intelligence: Context and implementation. *European Parliamentary Research Service*.
- Marques-Silva, J. (2023). Logic-based explainability in machine learning. In *Reasoning web. causality, explanations and declarative knowledge: 18th international summer school 2022, berlin, germany, september 27–30, 2022, tutorial lectures* (pp. 24–104). Springer.
- Marquis, P. (2015). Compile! *Proceedings of the AAAI Conference on Artificial Intelligence*, 29.
- Marzouk, R., Bassan, S., Katz, G., & de la Higuera, C. (2025). On the computational tractability of the (many) Shapley values. *arXiv preprint arXiv:2502.12295*.
- McCallum, J. C. (2023). Price and performance changes of computer technology with time. *U.S. Bureau of Labor Statistics – with minor processing by Our World in Data*. Available online: <https://ourworldindata.org/grapher/historical-cost-of-computer-memory-and-storage>.
- Narodytska, N., Kasiviswanathan, S., Ryzhyk, L., Sagiv, M., & Walsh, T. (2018). Verifying properties of binarized deep neural networks. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32.
- Ng, A. (2016). What artificial intelligence can and can’t do right now. *Harvard Business Review*, 9(11), 1–4.
- Okamoto, Y., Uno, T., & Uehara, R. (2005). Linear-time counting algorithms for independent sets in chordal graphs. *Graph-Theoretic Concepts in Computer Science: 31st International Workshop, WG 2005, Metz, France, June 23–25, 2005, Revised Selected Papers 31*, 433–444.
- Reiter, R. (1987). A theory of diagnosis from first principles. *Artificial intelligence*, 32(1), 57–95.
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). ”Why should I trust you?” Explaining the predictions of any classifier. *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, 1135–1144.
- Ruan, W., Huang, X., & Kwiatkowska, M. (2018). Reachability analysis of deep neural networks with provable guarantees. *arXiv preprint arXiv:1805.02242*.
- Shao, Z., Zhao, R., Yuan, S., Ding, M., & Wang, Y. (2022). Tracing the evolution of AI in the past decade and forecasting the emerging trends. *Expert Systems with Applications*, 209, 118221.
- Shapley, L. S., et al. (1953). A value for n-person games. *Princeton University Press Princeton*.
- Taylor, P. (2024). Amount of data created, consumed, and stored 2010-2023, with forecasts to 2028. *Statista*. Available online: <https://www.statista.com/statistics/871513/worldwide-data-created/>.
- Tjoa, E., & Guan, C. (2020). A survey on explainable artificial intelligence (XAI): Toward medical XAI. *IEEE transactions on neural networks and learning systems*, 32(11), 4793–4813.
- Van den Broeck, G., Lykov, A., Schleich, M., & Suciu, D. (2022). On the tractability of SHAP explanations. *Journal of Artificial Intelligence Research*, 74, 851–886.